National University of Computer and Emerging Sciences

Programming Fundamentals Lab (CL1002)

Date: 14-10-2024

Course Instructor

Ramsha Iqbal

Lab Mid Exam (A)

Total Time: 120 minutes

Total Marks: 20

Total Questions: 03

Semester: FL-2024

Campus: Karachi

Dept: Computer Science

Submission Instructions:

- You must comment your student ID on top of each file. (Line#1 of your code).
- Name the file for each question according to Roll_No e.g. **k24-xxxx_Q1.c, k24- xxxx _Q2.c** etc.
- Submission is via a client software so open the application present on the Desktop.
- Enter your username as **24K-xxxx** and its assigned password (**Default is Fast1234**).
- Submission is timed so after the time no submission will be accepted.

Syed Muhammad Taha — Student Name | 24K 0706 — Roll No | BCS 1G — Section | Student Signature

CLO # 1: Understand and Analyze flowcharts, PAC (Process-Activity-Control) charts, and IPO (Input-Process-Output) models to represent system workflows and these diagrams into algorithm and pseudocode implementation.

Q1. [5 marks] Conver the following PAC chart to C-code

Column	Detail
Given Data (Inputs)	**Starting number**
Required Results (Outputs)	**Number of steps to reach 1**
Processing	**-Initialize a step counter starting from zero. Repeat the process until the number becomes 1: -If number is even, divide by 2 -If number is odd, multiply by 3 and add 1 -Increment the step counter after each operation. -End the process once the number reaches 1.**
Conditions	**Number > 1; Check if even or odd**

10 → 5 → 16 → 8 → 4 → 2 → 1
1 2 3 4 5 6

11 → 34 → 18 → 9 → 28 → 14 → 7 → 22 → 11 → 34 → 18 → 9 → 28 → 14
1 2 3 4 5 6 7 8 9 10 11 12 13

CamScanner

CLO # 2: Gain hands on experience in writing code that provides the use of logical and bitwise operators to perform efficient data manipulation and apply decision and nested decision structures in control flow to create dynamic, condition-based logic within C-code.

Q2. [7 marks] You are given the task of creating a Simple Library Management System for a local library filled with books and friendly librarians. This library has only five books available for checkout, and the librarians need to manage the check-in and check-out process smoothly, like a game of tag. Each book can either be borrowed by a reader or returned to its spot on the shelf. But watch out—once a book is checked out, it goes missing until someone brings it back! Your program in C should have an easy-to-use interface that lets the library staff quickly check books in and out, making sure no book gets double-checked out. If two people try to take the same book, it could lead to a funny mix-up! With simple prompts and clear messages, your system will help the librarians keep everything organized and make the library a happy place for everyone who loves to read.

Sample Input & Output

```
Library Management System
1. Check Out a Book
2. Return a Book
3. Exit Choose an option: 1
Available books:
1. Book 1
2. Book 2
3. Book 3
4. Book 4
5. Book 5

Enter the number of the book you want to check out: 2
You have successfully checked out Book 2.

Library Management System
1. Check Out a Book
2. Return a Book
3. Exit
Choose an option: 1

Available books:
1. Book 1
3. Book 3
4. Book 4
5. Book 5
Enter the number of the book you want to check out: 4
You have successfully checked out 'Book 4'.
```


CamScanner

CLO # 3: Understand and implement code utilize loops for iteration and arrays for data storage and manipulation, demonstrating efficient traversal and management of elements in C-code, while optimizing control flow and enhancing program efficiency.

Q3. [8 marks] You are tasked with classifying positive integers into distinct categories: abundant, deficient, or perfect, based on their mathematical properties, while utilizing arrays to store the proper divisors of each integer evaluated. An abundant number is one whose sum of proper divisors exceeds the integer itself; for example, 36 is classified as abundant because its proper divisors (1, 2, 3, 4, 6, 9, 12, 18) sum up to 55, which is greater than 36. In contrast, a deficient number has a sum of proper divisors that is less than the integer; for instance, 27 is categorized as deficient because the sum of its proper divisors (1, 3, 9) is 13, significantly less than 27. A perfect number, on the other hand, has a sum of proper divisors that exactly equals the integer itself; for example, 28 is identified as perfect since its proper divisors (1, 2, 4, 7, 14) sum to exactly 28.

Write a program in C that prompts the user to input a positive integer, finds all proper divisors of the integer, and stores them in an array. After calculating the sum of the proper divisors, classify the integer as abundant, deficient, or perfect based on the criteria provided, and display the integer along with its classification and the list of proper divisors.

Sample Inputs	Sample Outputs
36	Abundant Factors are: 1 2 3 4 6 9 12 18 Sum is: 55
27	Deficient Factors are: 1 3 9 Sum is: 13
28	Perfect Factors are: 1 2 4 7 14 Sum is: 28
97	Deficient Factors are: 1 Sum is: 1
100	Abundant Factors are: 1 2 4 5 10 20 25 50 Sum is: 117
6	Perfect Factors are: 1 2 3 Sum is: 6

CamScanner